

Protocolo de imágenes BlackWall

Proyecto Servidor Asíncrono

Ciencias de la Computación VIII

Jorge Daniel Cuevas Mejia Carnet 22000138

Santiago Maldonado Barrios Carnet 19003827

1. Propósito del proyecto

BlackWall permite explorar imágenes grandes sin descargarlas completas en el navegador. El usuario elige una imagen, se acerca y se desplaza. El servidor lee las piezas de esa zona, llamadas mosaicos, y las envía primero como una previa ligera y después como detalle final.

El protocolo usa HTTP para intercambiar vistas, mosaicos y confirmaciones. Se probó con un PSB de aproximadamente 24 GB y PNG oficiales de aproximadamente 55 y 93 GB. El objetivo es mantener visible la zona elegida, reutilizar piezas y recuperar las que no se confirmaron.

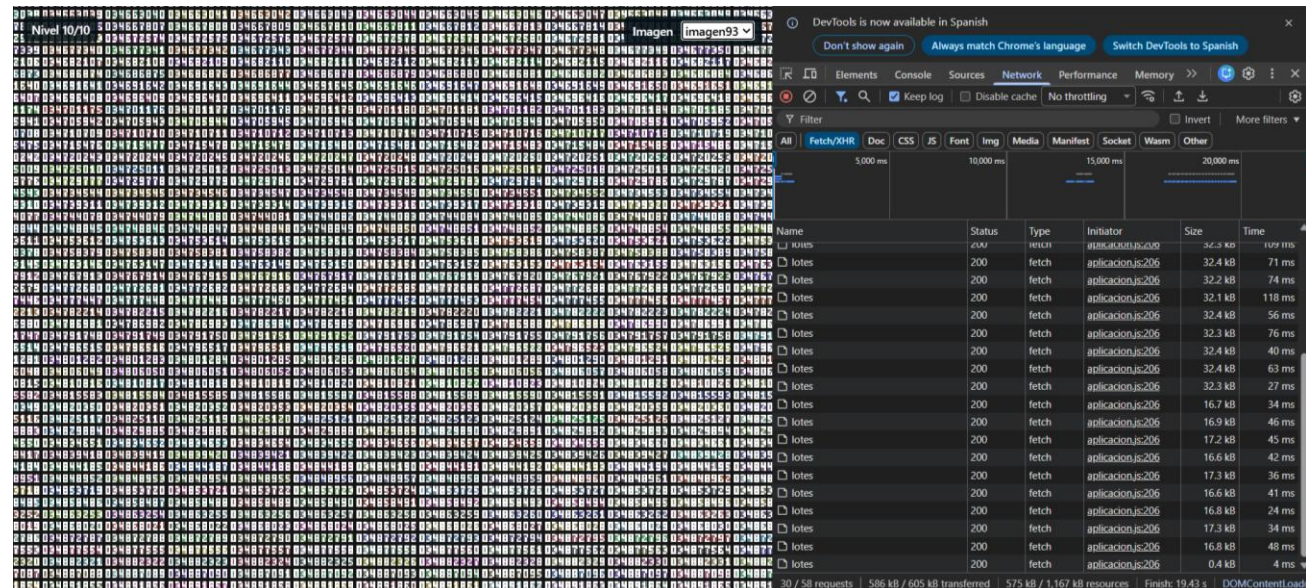


Imagen de 93 GB al nivel 10/10. Network muestra las peticiones que entregan los mosaicos de la zona visible. Sus tamaños corresponden a esas respuestas, no al original de 93 GB.

2. Qué pidió el proyecto y cómo se resolvió

El proyecto requiere un cliente y un servidor asíncronos para explorar regiones y niveles de imágenes grandes. También exige recuperación de errores, funcionamiento con red lenta y números legibles en las imágenes oficiales. Los algoritmos se explican con pseudocódigo y se distinguen sus adaptaciones de las propuestas originales.

Java mantiene las imágenes y las sesiones. El navegador calcula la zona visible, solicita mosaicos y los dibuja. Cada pieza se lee y codifica cuando le toca enviarse. Así se evita preparar toda la imagen y se puede descartar el trabajo de una zona que el usuario ya dejó atrás.

```
PS C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes> Set-Location -LiteralPath 'C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes'
PS C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes> powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\Compilar.ps1
[OK] Nucleo compilado.
PS C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes> java -cp 'construccion\clases' blackwall.servidor.IniciarServidor --pue
rto 8080 --imagen 'imagen24=C:\Users\santv\Downloads\eso1242a.psb' --imagen 'imagen55=C:\Users\santv\Downloads\Imagen-55GB-comprimida\055-843-000-80450114.p
ng' --imagen 'imagen93=C:\Users\santv\Downloads\Imagen-93GB-Comprimida\093-494-000-120123412.png'
Servidor disponible en http://127.0.0.1:8080; 3 imagen(es):
    imagen24: 108199x81503, formato PSB.
    imagen55: 136325x136325, formato PNG.
    imagen93: 176393x176393, formato PNG.
```

La terminal confirma que Java compiló y que el servidor inició. El catálogo muestra dimensiones y formatos de las tres imágenes disponibles.

3. Arquitectura y recorrido de una vista

Al abrir el visor, el navegador crea una sesión y consulta el catálogo. Después envía la imagen, el nivel y la región visible. El servidor programa las piezas que faltan y el cliente pide lotes hasta recibirlas y confirmarlas. Si cambia de imagen, nivel o zona, aumenta el número de vista. Ese número permite ignorar respuestas antiguas.

`ServidorHttpConcurrente` atiende peticiones con hilos virtuales. Mientras una espera por disco o HTTP, otras pueden avanzar. La lectura de disco sigue siendo bloqueante. El navegador usa fetch asíncrono para que los controles respondan durante la descarga.

Cada sesión conserva su vista, sus confirmaciones y sus medidas de red. `SesionTransferencia` sincroniza los cambios para no mezclar peticiones de la misma sesión. Cambiar de vista cancela el trabajo anterior y reinicia sus secuencias, pero conserva las medidas de congestión. Los lectores comparten metadatos e índices, no el canal de cada lectura.

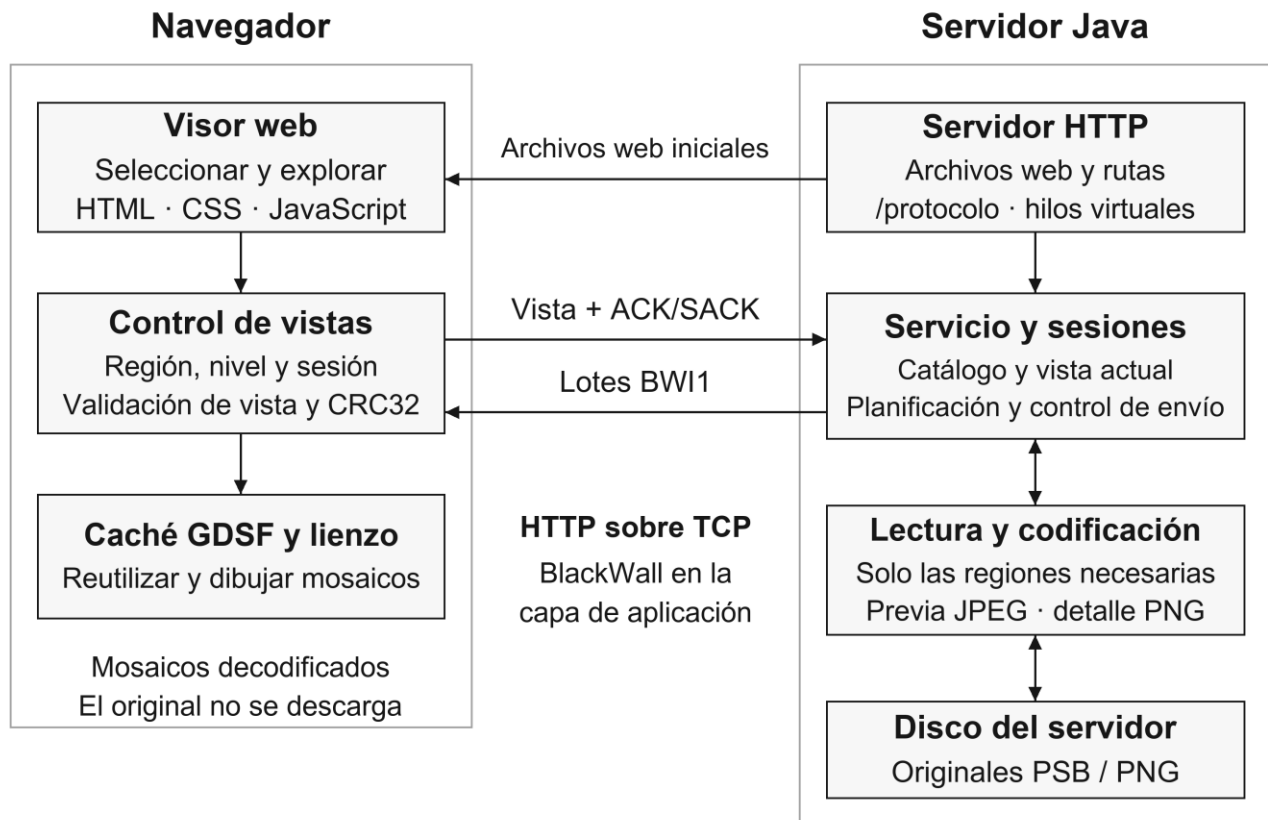


Diagrama 1. El navegador solicita regiones y el servidor lee y codifica sus mosaicos. Tanto la interfaz inicial como el protocolo usan HTTP sobre TCP. El original permanece en el disco del servidor.

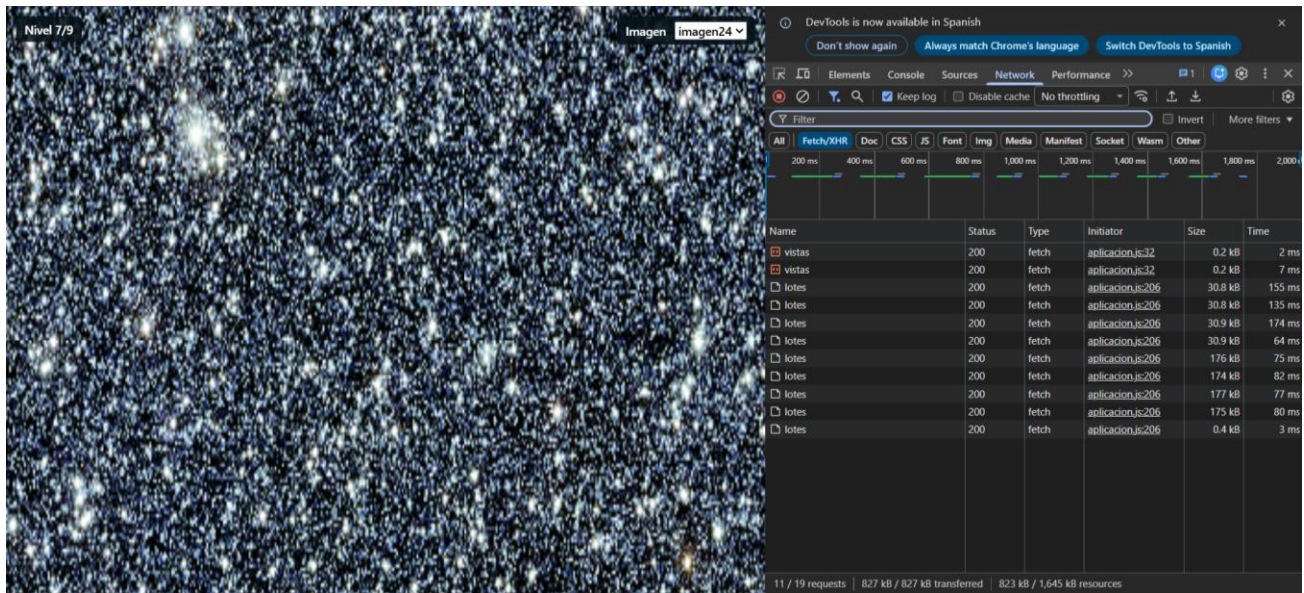


Imagen de 24 GB. vistas solicita la zona elegida y lotes devuelve sus mosaicos. Las respuestas 200 indican peticiones atendidas correctamente.

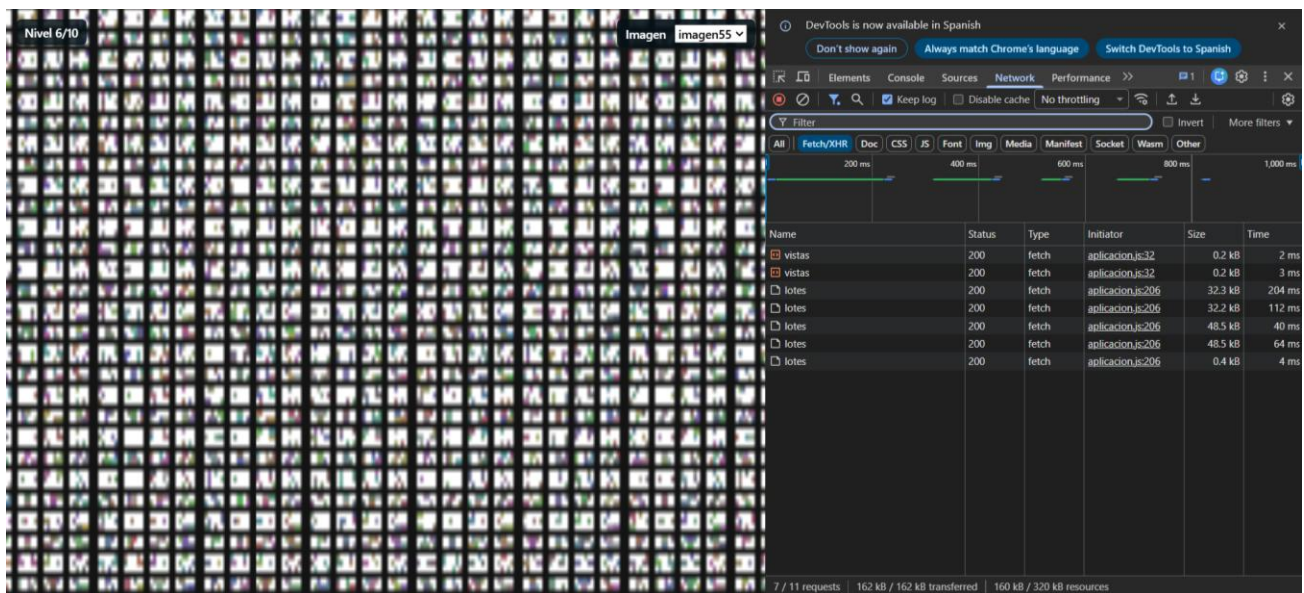


Imagen de 55 GB al nivel 6/10. Los tamaños en kB de lotes muestran cuánto se recibió para esa zona. No representan el tamaño de la imagen completa.

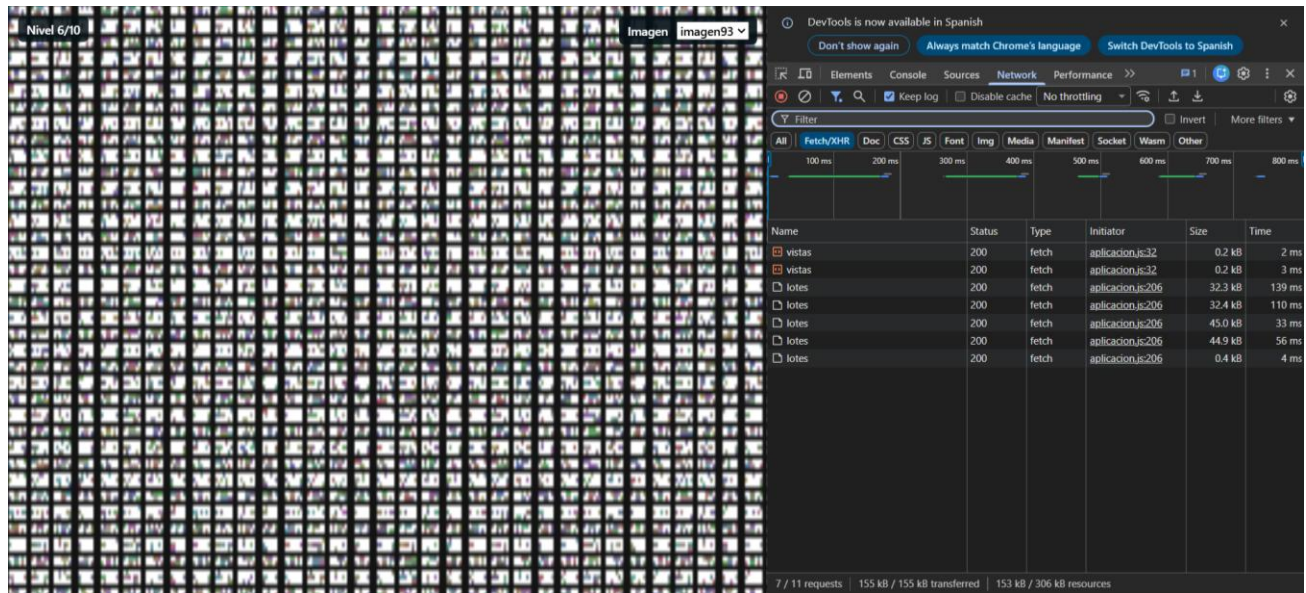


Imagen de 93 GB al nivel 6/10. Las peticiones vistas llevan parámetros y los lotes llevan las piezas necesarias para dibujar la zona.

El viewport es la zona de la imagen visible en pantalla. El navegador limita el desplazamiento a los bordes de la imagen. La región se describe con x, y, ancho y alto en coordenadas del original. Se actualiza al arrastrar y al cambiar el zoom. El acercamiento toma el cursor como referencia para mantener la zona elegida. El servidor usa esos datos para localizar las piezas que cruzan la vista.

PlanificadorRegiones divide la zona en mosaicos de hasta 256 píxeles por lado de salida. Piezas menores generan más peticiones y piezas mayores incluyen más área fuera de la vista. El planificador rechaza más de 64 posiciones y el servicio rechaza más de 128 trabajos, contando previas, finales y predicciones.

Cada mosaico se identifica por imagen, nivel, columna, fila y calidad. Al alejarse, el lector toma muestras más separadas para cubrir más área. El nivel máximo lee los píxeles originales de la región. El número de secuencia ordena las tramas dentro de cada vista.

Pseudocódigo 1. Recorrido principal, basado en aplicacion.js y ServicioProtocoloImágenes.java

```

FUNCTION mostrarVista(imagen, region, nivel)
    vista = vista + 1
    enviarVista(imagen, region, nivel, cache)
    WHILE faltanMosaicos(vista)
        lote = pedirLote(ACK, SACK, ventana)
        FOR EACH trama IN lote
            IF vistaValida(trama) AND crcValido(trama)
                guardarYMostrar(trama)
                confirmar(trama.secuencia)
            END IF
        END FOR
    END WHILE
END FUNCTION

```

4. Catálogo de mensajes y formato de datos

4.1 Solicitudes del navegador

El navegador crea una sesión, obtiene el catálogo y solicita la zona visible. El servidor indica los trabajos pendientes de previa, detalle final o predicción. Si no quedan trabajos, el navegador dibuja desde su caché sin pedir lotes.

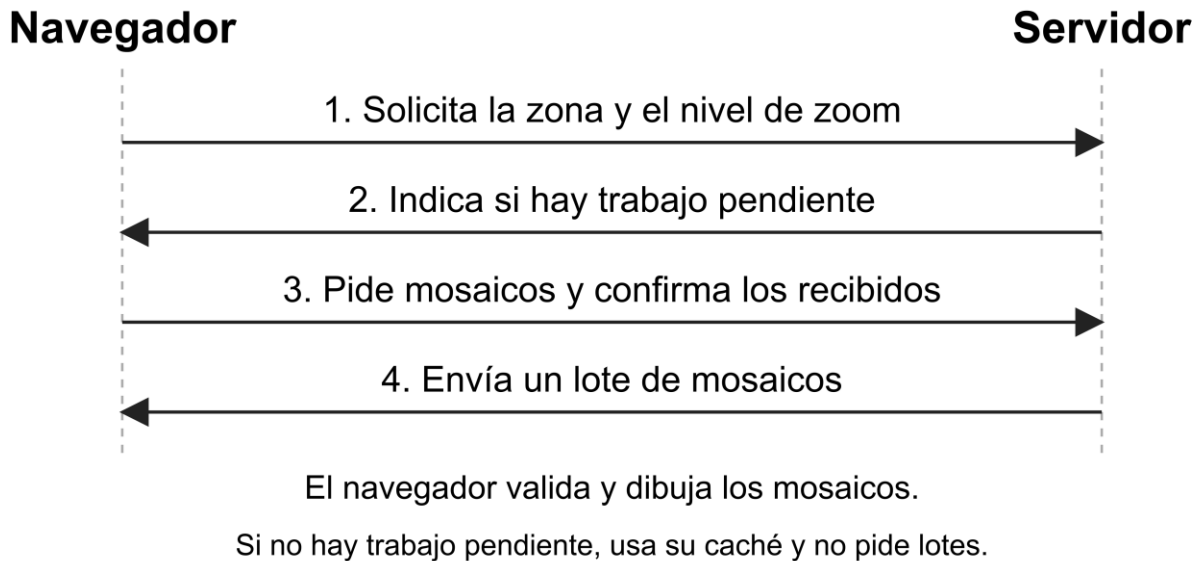
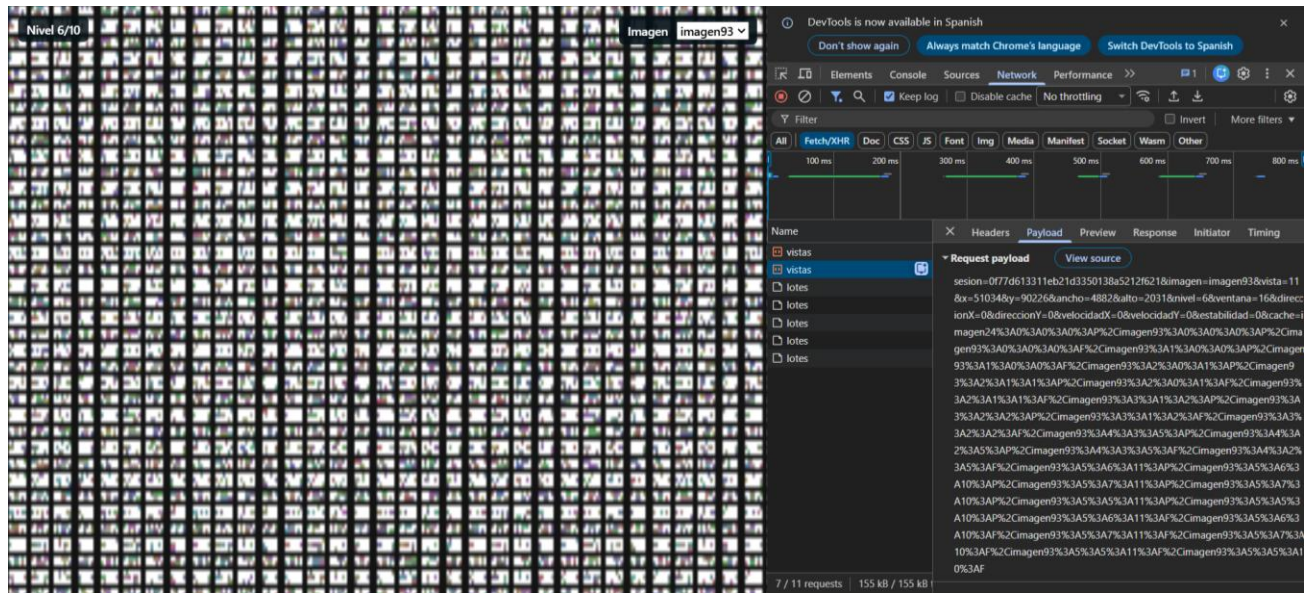


Diagrama 2. Tras abrir la sesión, el navegador solicita una vista. Los pasos 3 y 4 se repiten mientras queden mosaicos por recibir o confirmar.

Cada petición de lotes confirma lo recibido en la vista actual. ACK indica la última secuencia recibida sin huecos. SACK señala las recibidas después de un hueco. El cliente comprueba la vista y CRC32 y decodifica el mosaico antes de confirmar. Si falla, deja la trama sin confirmar para recuperarla. Estas confirmaciones pertenecen a BlackWall, no a TCP.

Las solicitudes usan pares nombre=valor. La vista incluye región, nivel, movimiento y claves de caché. El navegador anuncia una capacidad fija de 16 tramas y demoraAck=0.

X-Estado-Protocolo informa del envío. GET /protocolo/estado consulta ese estado y GET /salud comprueba si el servidor responde.



Payload muestra lo que el navegador pide para la imagen de 93 GB. sesion y vista identifican la solicitud. x, y, ancho, alto y nivel indican la zona y el acercamiento. cache lista claves de piezas guardadas para evitar reenvíos. El texto largo contiene esas claves, no los píxeles de la imagen.

4.2 Respuesta binaria

El lote empieza con cuatro bytes que indican la cantidad de tramas. Antes de cada trama, otros cuatro indican su longitud. Los enteros usan big-endian. Cada trama contiene una cabecera BWI1 de 32 bytes, un identificador UTF-8 de hasta 512 bytes y contenido JPEG o PNG de hasta 4 MiB. La tabla muestra posiciones desde el inicio de la trama.

Campo	Offset	Bytes	Interpretación
Marca	0	4	BWI1
Versión	4	1	1
Prioridad	5	1	Ordinal de PrioridadDato
Secuencia	6	8	Número positivo dentro de la vista
Vista	14	8	Identificador de la vista
Longitud del ID	22	2	Bytes UTF-8 del identificador
Longitud de contenido	24	4	Bytes del mosaico codificado
CRC32	28	4	CRC del contenido JPEG o PNG

La prioridad 0 corresponde a previa visible, 1 a final visible, 2 a previa predictiva y 3 a final predictiva. Esta última está definida en el formato, pero el planificador solo genera previas predictivas. CRC32 comprueba el contenido, no la cabecera, y no autentica ni cifra. Java valida el formato completo. El navegador comprueba marca, vista y CRC, y decodifica el contenido como JPEG o PNG según su clave.

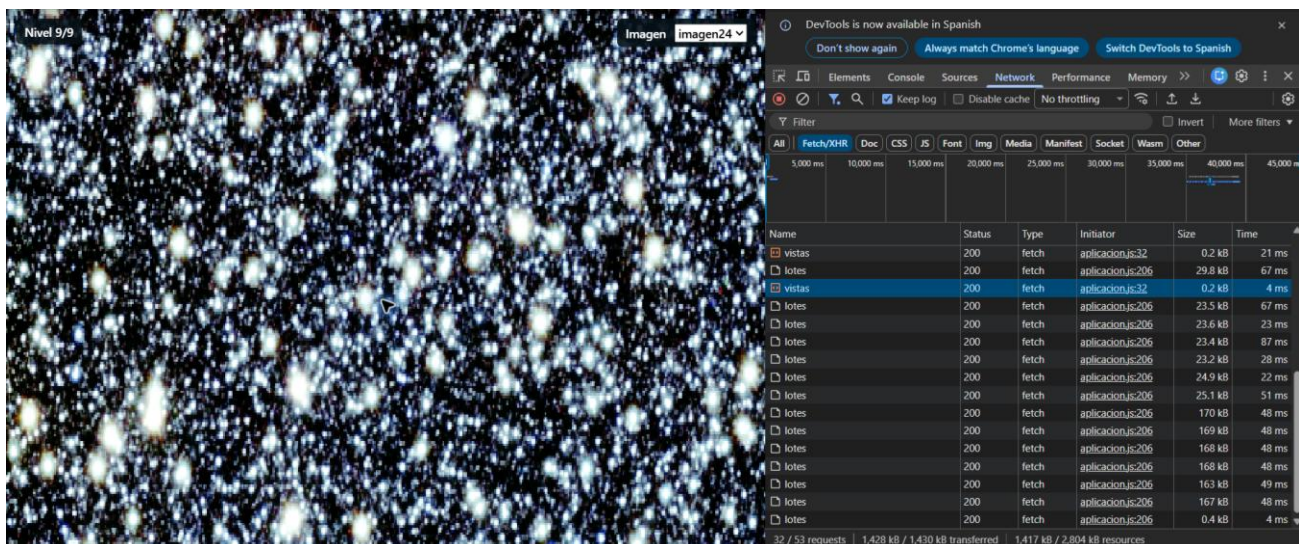
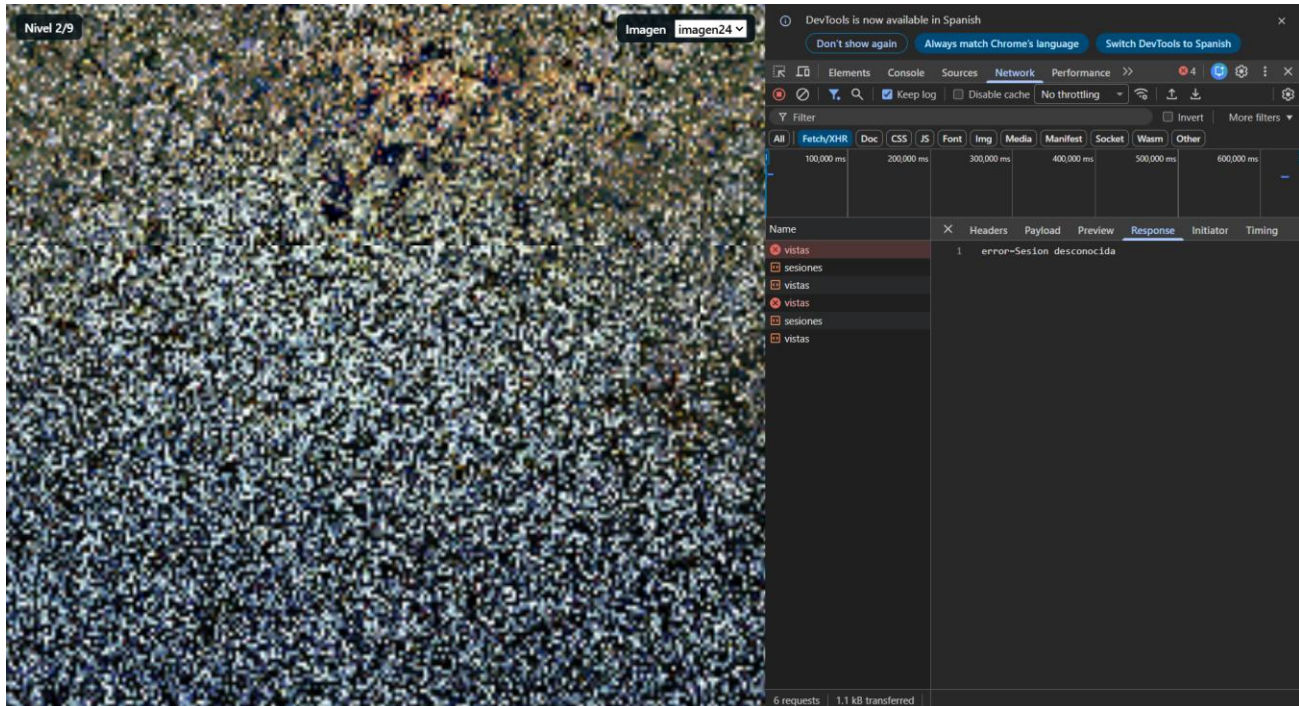


Imagen de 24 GB al nivel 9/9. Los lotes entregan mosaicos de esa región. El estado 200 confirma que se atendió la petición, no que ya se haya dibujado todo el detalle.

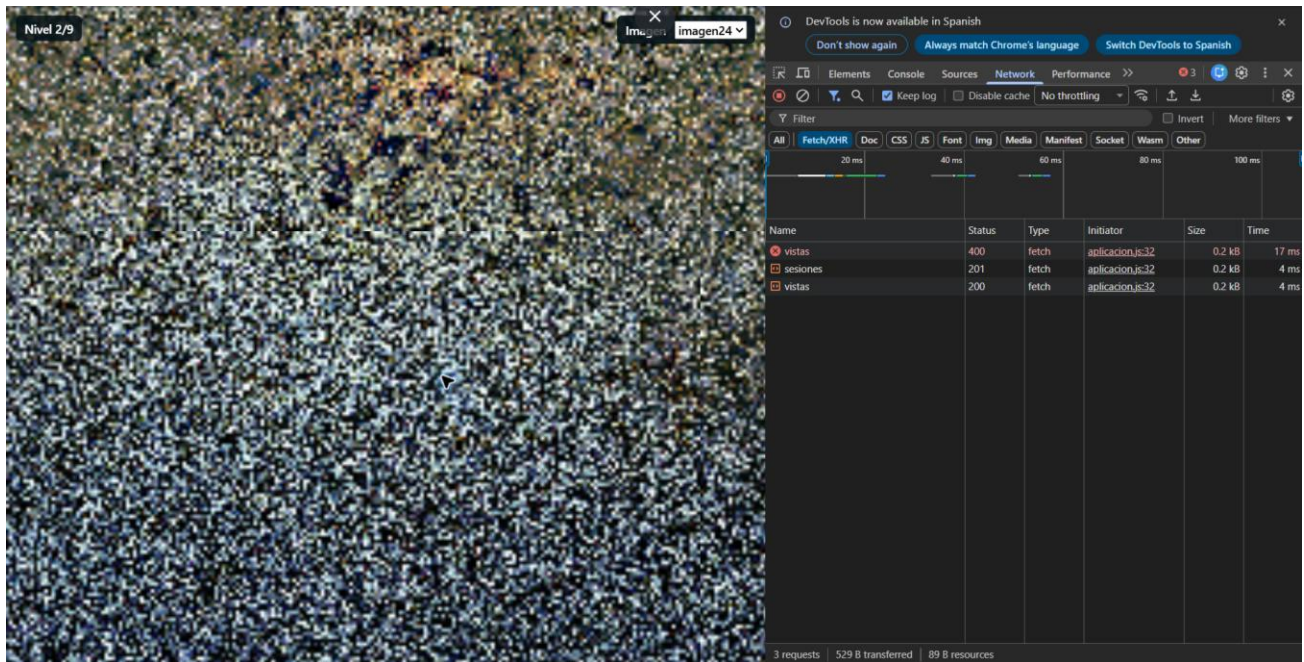
4.3 Errores y estados especiales

Una solicitud mal formada, una región o nivel inválido o una sesión desconocida devuelve 400. Un fallo interno de lectura devuelve 500. Se rechazan cuerpos mayores de 1 MiB. Crear una sesión con un método distinto de POST devuelve 405. Un lote de una vista anterior llega vacío y no modifica las confirmaciones de la actual.

GestorSesiones revisa cada minuto y elimina sesiones con más de diez minutos sin actividad. Ante el error de sesión desconocida, el cliente crea una sola sesión nueva y vuelve a solicitar la zona sin recargar. Otros fallos de red conservan la sesión y se reintentan con una espera creciente de hasta ocho segundos. La limpieza no fija un máximo de usuarios ni de memoria total.



Response muestra error=Sesion desconocida. El servidor ya no reconoce ese identificador y el cliente necesita crear otra sesión. Esta captura muestra el error.



La recuperación aparece como vistas con 400, sesiones con 201 y otra vista con 200. Esa secuencia muestra que la solicitud se retomó con una sesión nueva.

4.4 Cómo leer Network y el estado del protocolo

Network muestra peticiones HTTP, tiempos y bytes transferidos. X-Estado-Protocolo informa del trabajo pendiente y los reenvíos. rttMs mide hasta que el servidor recibe la confirmación del cliente. No equivale al tiempo de una petición en Network.

Al volver a una zona, mosaicos=0 y la ausencia de nuevos lotes indican reutilización de caché. HTTP 200 confirma una petición correcta, no que todo el detalle esté dibujado.

5. Procesamiento de las imágenes

LectorImagenRegional elige cuánto detalle leer según el nivel. Al alejarse aumenta el intervalo entre muestras. No guarda copias completas de cada nivel. Antes de leer limita la salida a 1048576 píxeles. Este límite controla esa imagen de salida, no toda la memoria del proceso.

La división en mosaicos se implementó en PlanificadorRegiones, sin una librería externa de pirámides. El lector usa acceso directo para PSB y un lector especializado para PNG con más de 2147483647 píxeles. ImageIO se usa para leer otros formatos compatibles, no para dividir la imagen. Los PNG oficiales gigantes usan el lector especializado, porque cargar sus píxeles completos excedería la memoria disponible.

5.1 Lectura regional del PSB

LectorPsbRegional admite PSB RGB de ocho bits, tres canales y sin compresión. Verifica su estructura y localiza el inicio de los píxeles. El archivo guarda los canales rojo, verde y azul por separado. Esto permite calcular qué posiciones de disco corresponden a la región.

El lector toma segmentos de las filas elegidas en cada canal, selecciona las columnas según el nivel y combina los colores. En memoria quedan la salida y los segmentos de fila necesarios. No se carga el PSB completo. Los segmentos ocupan memoria adicional al límite de salida.

5.2 Índice y lectura del PNG gigante

LectorPngGigante aprovecha la estructura de los PNG oficiales. Admite RGB de ocho bits, sin entrelazado, bloques IDAT de 8192 bytes salvo el último y DEFLATE almacenado. Las filas leídas deben usar filtro cero. Estas condiciones permiten acceder a los píxeles sin descomprimir todo lo anterior. Otros PNG pueden requerir otro lector.

En la primera lectura construye un índice de posiciones recorriendo cabeceras y saltando el contenido. Después calcula dónde está cada fila de la región, busca su bloque con una búsqueda binaria y lee los segmentos necesarios. El índice evita recorrer desde el inicio cada vez que se cambia de zona.

El índice .bwi se guarda en datos/indices para reutilizarlo al reiniciar. Se identifica con SHA-256 de la ruta y se valida con tamaño, fecha y estructura. Si no es válido, se reconstruye. Si falla su guardado, se usa en memoria. La primera vista puede tardar más por esta construcción. El original debe mantenerse estable, porque tamaño y fecha no garantizan que su contenido sea idéntico.

Pseudocódigo 2. Acceso regional en LectorPngGigante.java

```
FUNCTION leerRegionPNG(region, submuestreo)
  IF noHayIndiceEnMemoria()
    indice = cargarIndicePersistente()
    IF NOT metadatosYEntradasValidos(indice)
      indice = recorrerCabecerasDeflateAlmacenadas()
      validarLongitudTotalDePíxeles(indice)
      intentarGuardarIndiceConTemporal(indice)
    END IF
    conservarIndiceEnMemoria(indice)
  END IF
```

```
FOR EACH filaSeleccionada IN region
  validarFiltroCero(filaSeleccionada)
  bloque = buscarBinario(indice, posicionDePixeles(filaSeleccionada))
  segmento = leerIdatRegional(bloque, region)
  copiarColumnasSubmuestreadas(segmento, salida)
END FOR
RETURN salida
END FUNCTION
```

5.3 Codificación y calidad del detalle

CodificadorMosaicos envía una previa JPEG de calidad 0.55 para mostrar la zona pronto y después un PNG sin pérdida con las muestras leídas. En el nivel máximo esas muestras son los píxeles originales. El zoom adicional solo amplía el dibujo. No añade resolución y la previa JPEG no sirve para evaluar el detalle final.

El navegador recibe todo el cuerpo HTTP antes de dibujar. Por eso cada respuesta prepara como máximo un mosaico nuevo. Separar previa y final permite mostrar la previa sin esperar ambos. El cliente pide lotes secuenciales y una respuesta también puede incluir retransmisiones.

6. Los cuatro algoritmos y su adaptación

BlackWall adapta cuatro algoritmos para decidir qué enviar, cuánto mantener en tránsito, qué reutilizar y qué recuperar. Las adaptaciones trabajan sobre mosaicos de imágenes y no reproducen completos los sistemas originales.

6.1 Flare y la prioridad de la región visible

Flare aporta la idea de anticipar la zona que verá el usuario [1]. PlanificadorRegiones envía primero previas visibles y después detalle final, ordenados por el área que cubren de la vista. Así lo que el usuario necesita ahora tiene prioridad sobre una zona que quizá visite después.

Si el movimiento mantiene una dirección estable, el planificador puede añadir hasta cuatro previas futuras. Exige estabilidad de al menos 0.75 y capacidad receptora disponible. Proyecta el movimiento con dos RTT, entre 0.15 y 0.8 segundos, sin avanzar más de un lado de mosaico por eje.

SesionTransferencia permite enviarlas solo cuando no hay tramas visibles pendientes de confirmar, la ventana de congestión admite al menos cuatro tramas y el RTT actual no supera 1.25 veces el mínimo. Si no se cumplen las condiciones, descarta la predicción. Así un intento de anticipación no deja la vista esperando trabajo opcional.

Pseudocódigo 3. Flare tal como se usa en PlanificadorRegiones.java

```
FUNCTION planificar(vista, imagen, RTTms)
  visibles = mosaicos(vista.region, vista.nivel)
  IF cantidad(visibles) > 64 THEN rechazarVista()
  lista = previas(visibles) + finales(visibles)
  ordenar(lista, prioridad, areaVisible)
  IF movimientoEstable(vista) AND ventanaReceptora > 0
    tiempo = limitar(2 * RTTms / 1000, 0.15, 0.8)
    futura = proyectarSinSuperarUnMosaico(vista, tiempo)
    agregarHasta4PreviasNuevas(lista, futura)
  END IF
  RETURN lista
END FUNCTION
```

6.2 PCC Vivace y el control de tramas en tránsito

La adaptación de PCC Vivace ajusta el máximo de tramas sin confirmar según demora y pérdidas [2]. ControlCongestionVivace empieza con dos y aumenta al recibir confirmaciones. Si el RTT sube más de un 25 % sobre el mínimo, reduce la ventana al 85 % y pasa a evaluar intervalos. Una pérdida la reduce al 70 %, sin bajar de dos.

La utilidad favorece la entrega y penaliza la demora creciente y las pérdidas. Si empeora, el control invierte el ajuste y reduce su paso. Los envíos respetan esa ventana, la capacidad receptora y un límite de 16 MiB de contenido en vuelo por sesión. Una ventana receptora de cero detiene los envíos.

Esta política controla tramas de BlackWall, no TCP. El navegador actual pide lotes secuenciales con un mosaico nuevo por respuesta, lo que limita el efecto de ampliar la ventana. Las pruebas comprueban sus ajustes, pero no demuestran una mejora de velocidad en Chrome ni parámetros óptimos para cualquier red.

Pseudocódigo 4. Ajuste de ventana basado en ControlCongestionVivace.java

```
FUNCTION alConfirmar(RTT, cantidad)
  actualizarRTT(RTT)
  IF fase = ARRANQUE
    IF RTT > 1.25 * RTTmin
      ventana = max(2, ventana * 0.85)
      fase = CONTROL
      reiniciarIntervalo()
    ELSE
      ventana = limitar(ventana + cantidad, 2, maxima)
      RETURN
    END IF
  END IF
  acumularConfirmaciones(cantidad)
  IF intervaloIncompleto() THEN RETURN
  tasa = ventana / RTT
  utilidad = potencia(tasa, 0.8)
  utilidad = utilidad - 0.6 * tasa * aumentoRelativoRTT()
  utilidad = utilidad - 2 * tasa * fraccionPerdidas()
  ajustarDireccionYPaso(utilidad, anterior)
  ventana = limitar(ventana + direccion * paso, 2, maxima)
  anterior = utilidad
  reiniciarIntervalo()
END FUNCTION
```

6.3 GreedyDual Size Frequency y la reutilización de mosaicos

GDSF decide qué mosaicos conservar para no descargarlos de nuevo [3]. cache-mosaicos.mjs guarda hasta 128 entradas y 64 MiB estimados. Al superar un límite, elimina la pieza de menor valor. Ese valor aumenta con el uso y el coste aproximado de volver a transferirla, y disminuye con la memoria que ocupa.

Cada dibujo aumenta la frecuencia de uso. El coste se aproxima con los bytes JPEG o PNG recibidos y la memoria con ancho por alto por cuatro. La edad recoge el valor de la última pieza expulsada para que las entradas antiguas no permanezcan por su uso pasado. El cliente comunica las claves que conserva. El servidor omite esas piezas y también una previa si ya existe su versión final.

El servidor usa otra caché, de hasta 16 MiB por sesión, para reutilizar piezas leídas y codificadas. Allí aplica LRU, que elimina la menos usada recientemente. No es GDSF ni la caché HTTP de Chrome, pues las respuestas llevan no-store. Los límites de caché no incluyen el lienzo, los buffers de lectura ni el índice PNG.

Pseudocódigo 5. Reemplazo usado en cache-mosaicos.mjs

```
FUNCTION guardar(id, mosaico, coste)
  bytes = ancho(mosaico) * alto(mosaico) * 4
  IF bytes <= 0 OR bytes > limiteMemoria THEN RETURN
  frecuencia = frecuenciaAnterior(id, 1)
  valor = edad + frecuencia * max(1, coste) / bytes
  eliminarVersionAnterior(id)
  guardarEnCache(id, mosaico, bytes, frecuencia, valor)
  actualizarMemoria()
  WHILE entradas > 128 OR memoria > 64 MiB
    victima = entradaConMenorValor(cache)
    edad = victima.valor
    eliminar(victima)
  END WHILE
END FUNCTION
```

6.4 RACK-TLP y la recuperación de tramas

RACK-TLP permite recuperar tramas sin confirmar [4]. ACK confirma las secuencias continuas y SACK las recibidas después de un hueco. RACK reenvía una trama anterior cuando se confirmó una posterior y pasó el margen para tolerar reordenamiento. Ese margen se basa en un cuarto del RTT mínimo. Así no se interpreta de inmediato una llegada fuera de orden como pérdida.

TLP reenvía una vez la última trama para provocar una confirmación. Ese reenvío es la sonda. Solo se permite si queda una trama sin confirmar, no hay otras pendientes y ya existe un RTT suavizado. Espera dos RTT o hasta justo antes del RTO, el plazo que venza primero. No actúa si RACK o el timeout ya requieren reenvío. La sonda no cuenta por sí sola como pérdida. Los reenvíos por RACK o timeout sí reducen la ventana de Vivace.

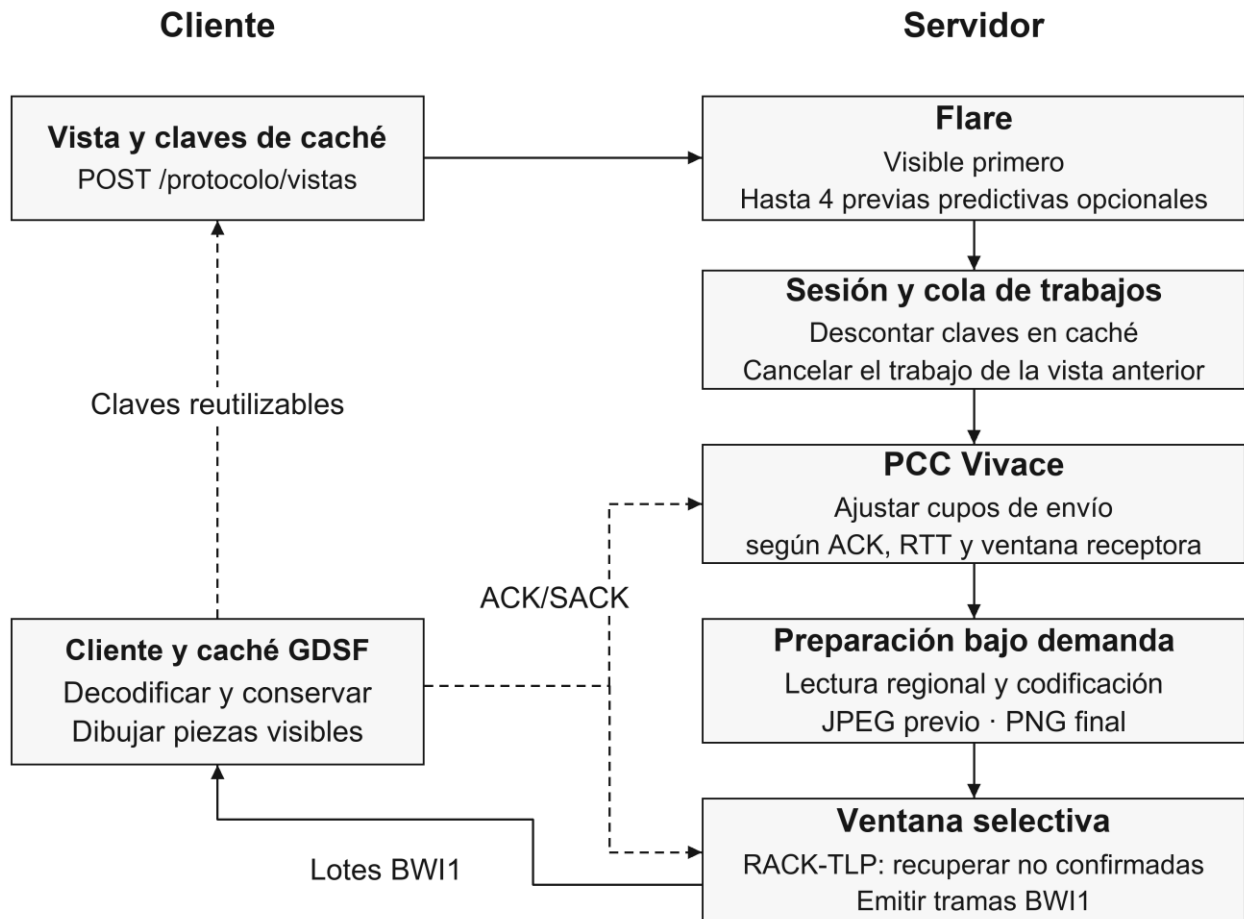
EstimadorRto calcula cuánto esperar antes de retransmitir usando el RTT suavizado y su variación. Empieza en 1000 ms y se mantiene entre 200 y 60000 ms. Cada timeout duplica la espera para no insistir demasiado. Una trama retransmitida no aporta una nueva muestra de RTT. El servidor revisa estos plazos al recibir otra petición de lote. Al cambiar de vista cancela las tramas anteriores. Esta recuperación es de BlackWall, no una modificación de TCP.

Pseudocódigo 6. Revisión de la ventana en VentanaTransmisionSelectiva.java

```
FUNCTION emitir(ahora)
    limite = min(ventanaReceptora, piso(ventanaCongestion))
    IF limite = 0 THEN RETURN listaVacia
    salida = listaVacia
    FOR EACH trama IN enviadas
        timeout = tiempoSinConfirmar(trama) >= RTO
        rack = posteriorConfirmada(trama) AND margenCumplido(trama)
        tlp = puedeSondearCola(trama) AND tiempoSondaCumplido(trama)
        tlp = tlp AND NOT timeout AND NOT rack
        IF timeout OR rack OR tlp
            reenviarYAgregar(salida, trama)
            IF tlp THEN marcarSondaEnviada(trama)
            IF timeout OR rack THEN registrarPerdidaEnVivace()
            IF timeout THEN duplicarRtoDentroDeLimites()
            IF cantidad(salida) >= limite THEN RETURN salida
        END IF
    END FOR
    enviarPendientes(salida, limite, 16 MiB)
    RETURN salida
END FUNCTION
```

6.5 Relación entre los cuatro algoritmos

SesionTransferencia coordina las decisiones. Flare ordena las piezas, GDSF evita pedir las conservadas, Vivace limita las tramas y RACK-TLP recupera las no confirmadas. Cambiar de vista descarta el trabajo anterior para que las respuestas tardías no alteren la zona nueva.



Adaptaciones del protocolo de aplicación; no sustituyen TCP.

Diagrama 3. Las líneas continuas muestran solicitudes y mosaicos. Las discontinuas llevan claves de caché y confirmaciones. Cada adaptación interviene en una decisión distinta [1]-[4].

```
PS C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes> powershell -NoProfile -ExecutionPolicy Bypass -File .\scripts\Probar.ps1
[OK] 7/7 grupos del nucleo superados.
[OK] Lectura regional, memoria acotada, planificacion y salida sin perdida superadas.
[OK] Integracion GDSF-servidor: bloque retenido evita envio; bloque expulsado se retransmite.
[OK] Integracion HTTP, lotes binarios, estado y concurrencia superadas.
[OK] Limpieza periodica de sesiones inactivas.
[OK] Adaptacion Vivace y control de flujo con ventana cero superadas.
[OK] Adaptacion Flare: cobertura, movimiento, preparacion diferida, cache, continuidad y prediccion.
[OK] GDSF: expulsión real en la entrada 129/128; sobrevive el bloque frecuente.
[OK] GDSF: expulsión real por 64 MiB; sobrevive el bloque costoso.
[OK] GDSF: envejecimiento, reemplazo y contabilidad de memoria.
[OK] Zoom 10→0→10 conserva la región; un arrastre cambia de zona y el recorrido inverso vuelve a ella.
[OK] Deteccion y renovacion unica de sesiones expiradas.
PS C:\Users\santv\Documents\Codex\2026-09-12\n\outputs\blackwall-protocolo-imagenes>
```

Las líneas OK muestran comprobaciones superadas de planificación, ventana, caché, recuperación y zoom. Verifican los casos ejecutados. La legibilidad de los números se revisó aparte en Chrome.

7. Organización del código

Los componentes principales permiten seguir una solicitud desde el navegador hasta el archivo y localizar las decisiones explicadas en este documento.

Servidor y configuración

IniciarServidor.java lee la configuración y crea los servicios. ServidorHttpConcurrente.java expone las rutas y sirve la interfaz. ServicioProtocoloImágenes.java conecta el planificador, los lectores y las sesiones. ParametrosPetición.java interpreta y valida los campos recibidos.

Imagen y planificación

CatalogoImágenes.java y DescriptorImagen.java registran las rutas y los metadatos. RegionImagen.java, SolicitudVista.java y MosaicoSolicitado.java describen la zona pedida y sus piezas.

PlanificadorRegiones.java elige las piezas. LectorImagenRegional.java dirige la lectura a LectorPsbRegional.java, LectorPngGigante.java o ImagenIO. CodificadorMosaicos.java produce la previa y el detalle final.

Protocolo, congestión y sesiones

CodificadorTramas.java escribe y valida BWI1. PaqueteDatos.java y PrioridadDato.java representan su contenido y prioridad. ConfirmacionRecepcion.java y RangoSack.java describen las confirmaciones.

ControlCongestionVivace.java ajusta la ventana y EstimadorRto.java calcula la espera de recuperación. VentanaTransmisionSelectiva.java procesa ACK, SACK y retransmisiones. SesionTransferencia.java conserva y prepara los trabajos de cada vista. TrabajoMosaico.java difiere la lectura hasta que se necesita.

GestorSesiones.java crea y elimina las sesiones. EstadoVentana.java reúne los contadores consultables del protocolo.

Interfaz

interfaz.html y estilos.css definen los controles y el área de imagen. aplicacion.js coordina solicitudes y dibujo. cache-mosaicos.mjs administra GDSF, navegacion-zoom.mjs calcula el acercamiento y recuperacion-sesion.mjs renueva una sesión desconocida.

La rueda, los botones y el deslizador controlan el zoom. Las teclas + y - acercan o alejan, 0 vuelve a la vista general y F alterna pantalla completa, salvo al escribir en un campo. El visor ignora respuestas de imágenes anteriores y marca el detalle listo solo cuando tiene las piezas finales visibles.

Automatización y verificación

Makefile reúne los comandos de los scripts PowerShell para compilar, iniciar, probar, registrar imágenes y limpiar clases. `imagenes.example.txt` muestra el formato y `imagenes.local.txt` conserva las rutas del equipo, fuera del repositorio.

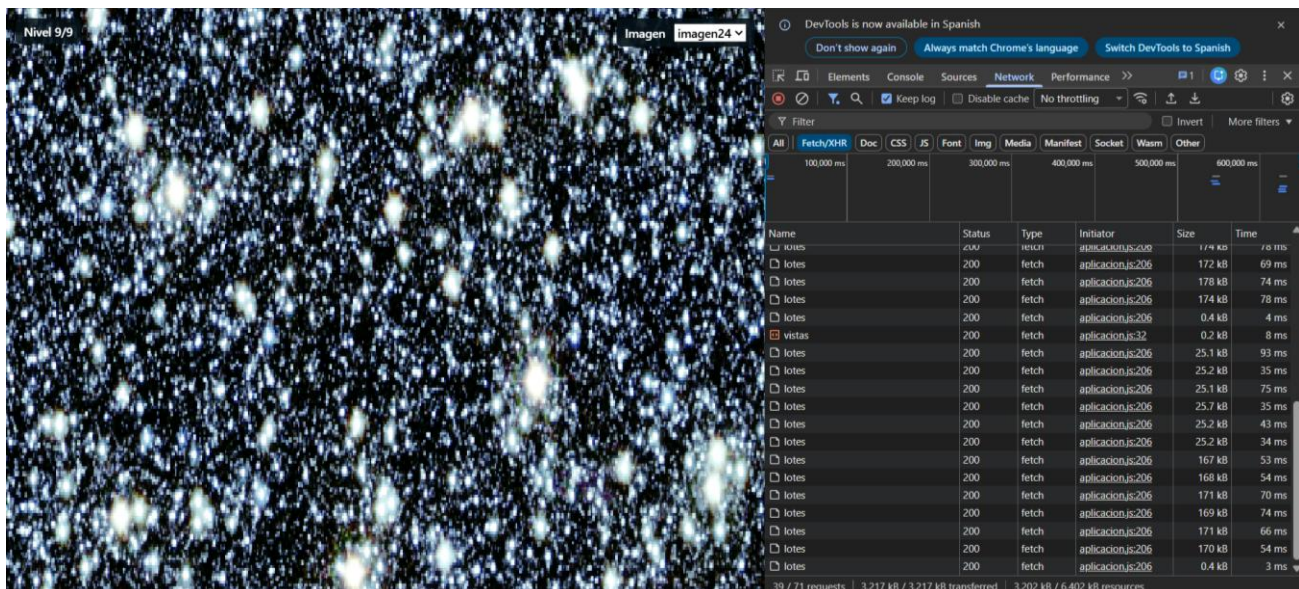
`Probar.ps1` ejecuta pruebas del núcleo, lectores, integración, sesiones y políticas. Los scripts del cliente comprueban caché, zoom y recuperación de sesión. `ProbarCambiosYPerdidas.mjs` simula un hueco y un cambio de vista. `ProbarRedLenta.mjs` emula demora y ancho de banda.

`PruebasPngGigante.java` y `PruebasTresImagenesReales.java` comprueban la lectura de las imágenes configuradas. Los informes `PRUEBAS-IMAGEN-24GB.md`, `PRUEBAS-IMAGEN-55GB.md` y `PRUEBAS-IMAGEN-93GB.md` registran las pruebas reales y sus condiciones.

8. Pruebas y resultados

`Probar.ps1` terminó con código cero, incluido el resultado de siete grupos de `PruebasNucleoProtocolo`. La batería comprueba tramas, recuperación, lectura regional, límites de salida, planificación, PNG sin pérdida, integración y sesiones concurrentes. También se probaron caché, zoom y renovación de sesión. Son comprobaciones de esos casos, no una medición de toda la memoria del servidor.

Con la imagen de 24 GB se llegó a 9/9 y se navegó con 500 Kbps, 3G, latencia alta y cambios rápidos de zona. Al volver a una región, Network mostró tres vistas, ningún lote y 504 bytes transferidos. Al bloquear lotes quedó detalle pendiente. Desbloquearlos permitió completarlo sin recargar. Esta fotografía no se usó para evaluar números.



La imagen de 24 GB al nivel 9/9 muestra nuevas vistas y lotes durante la navegación. Cambiar de zona genera una solicitud de región y nuevas piezas.

El PNG de 55 GB mide 136325 por 136325 píxeles. Se leyeron esquina, centro, borde y vista general. En Chrome se distinguieron números al nivel 10/10. El cliente de prueba recuperó un hueco simulado con una retransmisión. Con 400 Kbps y 2000 ms de latencia emulados llegó la previa antes del final y la revisita no pidió mosaicos. Chrome también se probó con red lenta, latencia alta y reutilización entre niveles.

The screenshot shows a web browser interface with a large image on the left and a network log on the right. The image is labeled 'Imagen imagen55' and 'Nivel 8/10'. The network log table on the right contains the following data:

Name	Status	Type	Initiator	Size	Time
index	200	fetch	aplicacion.js:206	46.2 kb	90 ms
lotes	200	fetch	aplicacion.js:206	48.5 kb	64 ms
lotes	200	fetch	aplicacion.js:206	0.4 kb	4 ms
vistas	200	fetch	aplicacion.js:32	0.2 kb	5 ms
lotes	200	fetch	aplicacion.js:206	30.9 kb	101 ms
lotes	200	fetch	aplicacion.js:206	31.0 kb	81 ms
lotes	200	fetch	aplicacion.js:206	40.3 kb	28 ms
lotes	200	fetch	aplicacion.js:206	39.7 kb	29 ms
lotes	200	fetch	aplicacion.js:206	0.4 kb	5 ms
vistas	200	fetch	aplicacion.js:32	0.2 kb	9 ms
lotes	200	fetch	aplicacion.js:206	28.6 kb	89 ms
lotes	200	fetch	aplicacion.js:206	28.7 kb	96 ms
lotes	200	fetch	aplicacion.js:206	28.1 kb	35 ms
lotes	200	fetch	aplicacion.js:206	28.3 kb	49 ms
lotes	200	fetch	aplicacion.js:206	29.8 kb	29 ms
lotes	200	fetch	aplicacion.js:206	30.8 kb	31 ms
lotes	200	fetch	aplicacion.js:206	29.4 kb	32 ms
lotes	200	fetch	aplicacion.js:206	30.3 kb	31 ms
lotes	200	fetch	aplicacion.js:206	0.4 kb	4 ms

23 / 39 requests | 539 kb / 539 kb transferred | 531 kb / 1,061 kb resources

Imagen de 55 GB al nivel 8/10. Los lotes corresponden a la zona elegida. Este nivel aún toma muestras separadas y no muestra todo el detalle original.



Imagen de 55 GB al nivel 10/10. Las cifras se distinguen en esta región porque se leen los píxeles originales y el detalle final se envía en PNG sin pérdida.

El PNG de 93 GB mide 176393 por 176393 píxeles. Se verificaron lectura regional y números legibles al máximo nivel. Alejarse y acercarse conservó la región solicitada. Un hueco simulado se recuperó con una retransmisión. Desbloquear lotes completó la misma zona sin recargar. Con red lenta y latencia alta llegó el detalle final. Una revisita indicó mosaicos=0 y no generó más lotes.

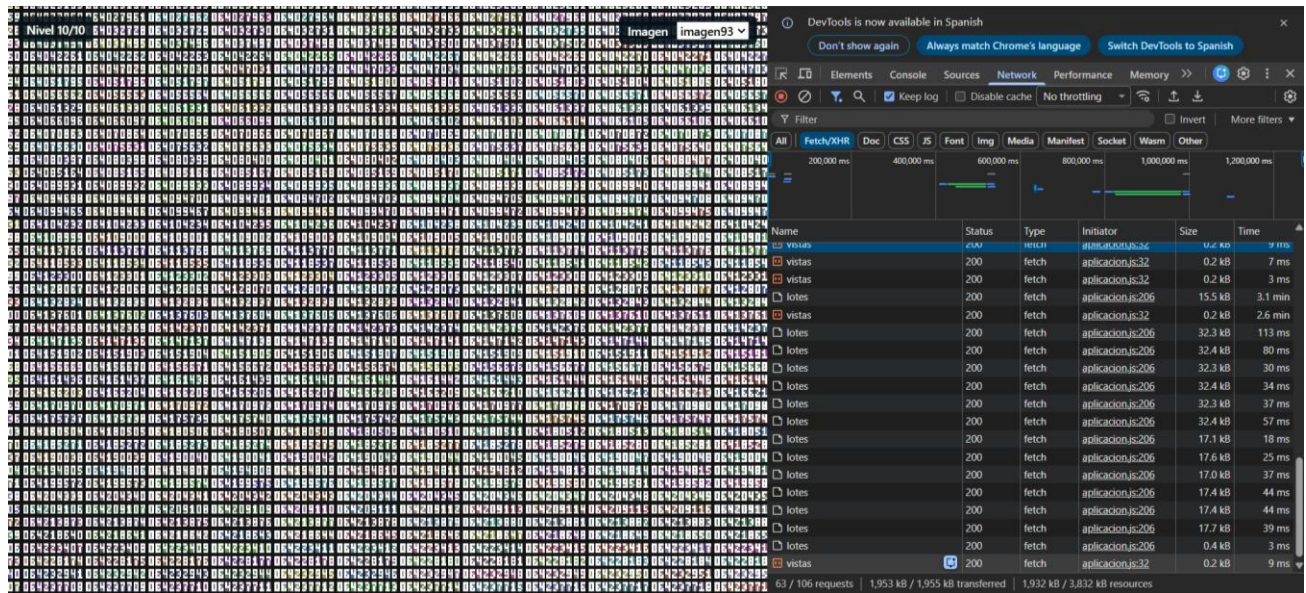


Imagen de 93 GB al nivel 10/10. Las peticiones vistas indican la región y los lotes entregan sus piezas. Network permite relacionar la navegación con lo recibido.



Detalle de la imagen de 93 GB al nivel 10/10. Las cifras de esta región son legibles. El PNG final conserva las muestras originales, a diferencia de la previa JPEG.

Con acercamiento adicional 3/3 se examinaron los bloques de píxeles de imagen93, sin suavizado. Esa ampliación no añade resolución. La observación corresponde a la región mostrada, no a todos los números del archivo.

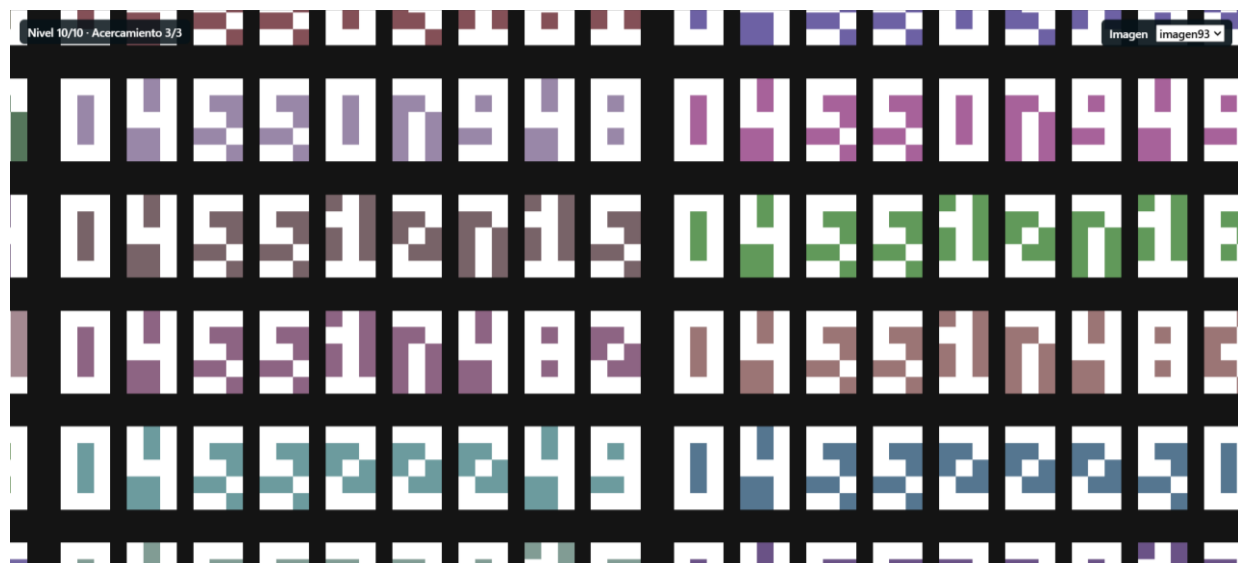


Imagen93 al nivel 10/10 y acercamiento adicional 3/3. Se ven las cifras y sus píxeles ampliados para examinarlos.

Los informes PRUEBAS-IMAGEN registran fechas y condiciones para reproducir estas observaciones. Los huecos se simularon en la aplicación y el bloqueo se hizo en DevTools. No prueban pérdidas reales de paquetes ni todos los fallos de Internet. No se comparó formalmente el rendimiento con otro visor.

9. Cómo ejecutar y demostrar el sistema

En Windows se necesitan PowerShell, GNU Make, JDK 21 o posterior y Node.js para las pruebas del cliente. Se copia `imagenes.example.txt` a `imagenes.local.txt` y se añaden identificadores y rutas absolutas. Los originales deben entregarse por separado o con enlaces accesibles. Ni los originales ni la configuración local se incluyen en el repositorio.

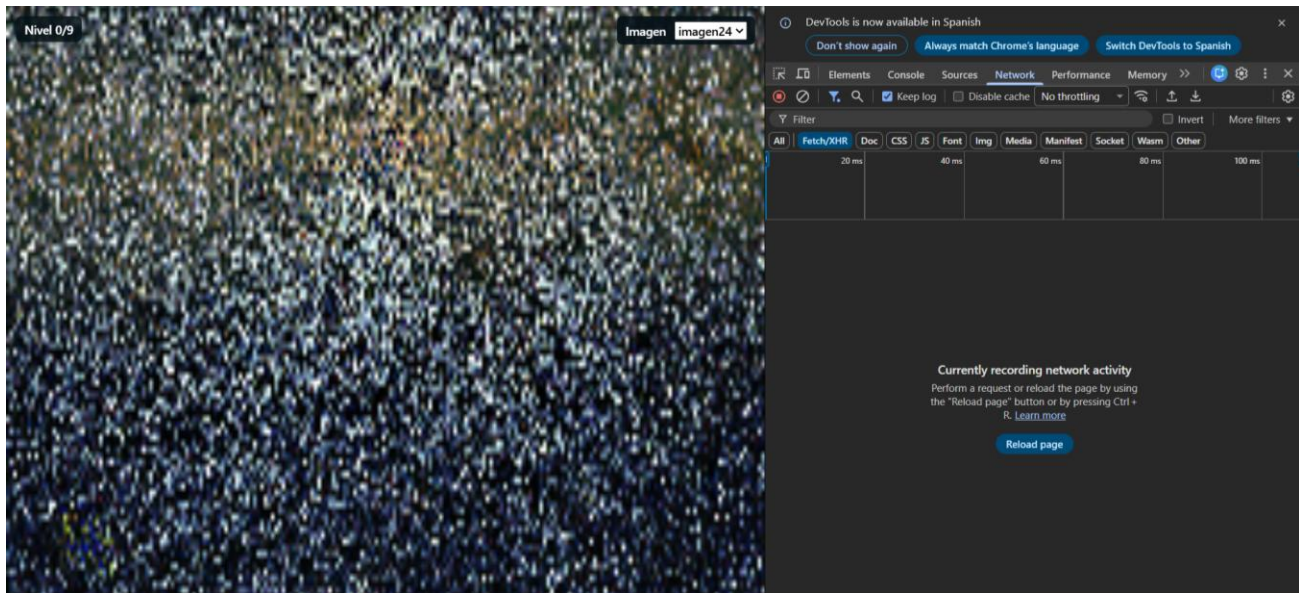
El servidor debe permanecer activo mientras se usa el visor. Tras registrar otra imagen se reinicia para actualizar el catálogo. Los comandos no modifican los originales.

El servidor escucha en 127.0.0.1 para acceso local, sin autenticación ni cifrado. Si el puerto está ocupado, se detiene la instancia anterior o se elige otro. Al reiniciar se renuevan las sesiones y se reutilizan los índices `.bwi` válidos.

Los comandos se ejecutan desde la raíz del proyecto.

- **make build**
Compila el núcleo Java. El script utiliza `javac --release 21`.
- **make server**
Recompila e inicia el servidor con las imágenes de `imagenes.local.txt` en el puerto 8080.
- **make server PORT=9090**
Inicia el servidor en otro puerto.
- **make add-image ID=imagenNueva FILE="C:/ruta/imagen.png"**
Registra otra imagen en el catálogo local.
- **make list-images**
Muestra el catálogo de imágenes registradas.
- **make test**
Ejecuta las pruebas automatizadas.
- **make clean**
Elimina solo las clases compiladas y conserva los índices `.bwi`.

Después de confirmar el arranque, se abre `http://127.0.0.1:8080` en Chrome. En Network se filtra Fetch/XHR y se limpia el registro antes de cada comprobación para separar las peticiones nuevas.



El registro está vacío antes de la prueba. Fetch/XHR filtra las peticiones del protocolo y No throttling indica que no se limita la red en ese momento. El registro vacío no demuestra una apertura sin transferencia.

Payload de vistas permite comprobar imagen, nivel, región y claves de caché. La respuesta indica cuántas piezas faltan. Al volver a una zona sin recargar ni cambiar de sesión, mosaicos=0 y la ausencia de lotes muestran reutilización. Para comprobar recuperación se bloquea lotes, se pide una zona nueva y se desbloquea. La zona debe completarse sin recargar. También se repite la navegación con red lenta y se revisan los números al nivel máximo.

10. Referencias

- [1] F. Qian, B. Han, Q. Xiao y V. Gopalakrishnan, "Flare: Practical Viewport-Adaptive 360-Degree Video Streaming for Mobile Devices," ACM MobiCom, 2018, pp. 99–114. DOI 10.1145/3241539.3241565. [Artículo completo publicado por el autor](#)
- [2] M. Dong et al., "PCC Vivace: Online-Learning Congestion Control," USENIX NSDI, 2018, pp. 343–356. [Artículo completo en USENIX](#)
- [3] L. Cherkasova, "Improving WWW Proxies Performance with Greedy-Dual-Size-Frequency Caching Policy," Hewlett-Packard Laboratories, informe HPL-98-69 (R.1), noviembre de 1998. [Informe completo conservado por la Universidad de Atenas](#)
- [4] Y. Cheng, N. Cardwell, N. Dukkipati y P. Jha, "The RACK-TLP Loss Detection Algorithm for TCP," IETF, RFC 8985, febrero de 2021. [Especificación completa en RFC Editor](#)
- [5] V. Paxson, M. Allman, J. Chu y M. Sargent, "Computing TCP's Retransmission Timer," IETF, RFC 6298, junio de 2011. [Especificación del cálculo del RTO en RFC Editor](#)
- [6] R. Fielding, M. Nottingham y J. Reschke, eds., "HTTP Semantics," IETF, RFC 9110, junio de 2022. [Especificación de solicitudes y respuestas HTTP en RFC Editor](#)
- [7] World Wide Web Consortium, "Portable Network Graphics (PNG) Specification (Third Edition)," recomendación W3C, 24 de junio de 2025. [Especificación completa del formato PNG en W3C](#)

- [8] P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," RFC 1951, mayo de 1996. Especificación de DEFLATE y sus bloques almacenados en RFC Editor
- [9] Adobe, "Adobe Photoshop File Formats Specification," noviembre de 2019. Especificación oficial de los formatos PSD y PSB